O Caminho Mais Rápido por Hyrule

Proposta de problema para a disciplina de Computação Básica, baseada no Problem Set 2 ("Fastest Way to Get Around MIT")

Documento submetido para aprovação.

Introdução

Neste problema os alunos resolvem um problema de otimização sobre um grafo. Em vez do campus do MIT, o mapa é o reino de Hyrule, de The Legend of Zelda: Breath of the Wild. Os 120 santuários do jogo são os pontos de viagem rápida do Link. O objetivo é encontrar a rota mais curta entre dois santuários, limitando ao mesmo tempo o perigo a que o Link se expõe pelo caminho, já que algumas regiões do mapa são perigosas.

A estrutura segue a do problem set original: o "limite de distância no exterior" é substituído por um "limite de perigo", o que mantém o mesmo tipo de raciocínio (dois pesos por aresta, uma função objetivo e uma restrição).

Ficheiros fornecidos

1. `graph.py`: estruturas de dados do grafo (Node, Edge, WeightedEdge, Digraph), que os alunos têm de usar.
2. `ps2.py`: esqueleto do código com as funções a implementar e os testes.
3. `shrines.csv`: os 120 santuários, com id, nome, região e posição sobre a imagem do mapa.
4. `edges.csv`: as arestas dirigidas do grafo, com origem, destino, distância e nível de perigo.
5. `botw_map.jpg` e `show_map.py`: imagem do mapa e script que desenha o grafo (apenas para visualização).

Como os dados foram construídos

Os dados foram gerados pelos scripts `scrape_shrines.py` (lista de santuários e regiões), `mark_shrines.py` (marcação das posições no mapa) e `edges_creator.py` (criação das arestas). Cada santuário liga-se aos seus vizinhos mais próximos, e foram acrescentadas ligações adicionais onde necessário para que **exista sempre um caminho entre quaisquer dois santuários**. O grafo resultante tem **120 nós e 233 arestas dirigidas**.

Cada aresta tem dois pesos: a **distância** (em píxeis do mapa) e o **nível de perigo**, que é o número de extremos da aresta (0, 1 ou 2) situados numa região perigosa. As arestas só podem ser percorridas no sentido indicado.

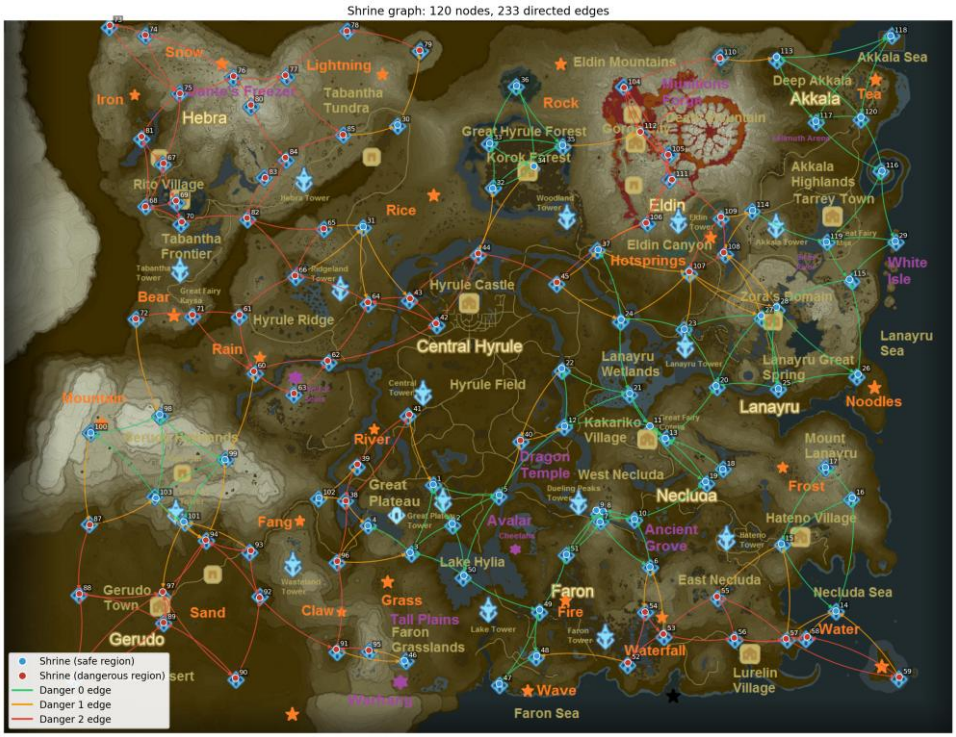


Figura 1. O grafo sobre o mapa (cor da aresta = nível de perigo).

Problema 1: Estruturas de Dados

Em `graph.py`, os alunos completam as classes `WeightedEdge` e `Digraph` de forma a que os testes no fim do ficheiro passem.

- `WeightedEdge` guarda os dois pesos (distância e perigo) e implementa `__str__`.
- `Digraph` implementa `add_node` e `add_edge`, que lançam `ValueError` quando o nó já existe ou quando uma aresta usa um nó que não está no grafo.

Problema 2: Construir o Mapa

2a. Modelação

Os alunos explicam, em comentário, o que representam os nós, as arestas e os pesos, e porque é que o grafo é dirigido.

2b. load_map

Implementar `load_map`, que lê os dois ficheiros CSV e devolve um `Digraph`. Cada santuário torna-se um `Node` e cada linha de arestas uma `WeightedEdge`. A distância é um número decimal e o perigo um inteiro.

2c. Teste do load_map

Os alunos criam um mini-mapa próprio (pelo menos 3 nós e 3 arestas), carregam-no e verificam se o grafo está correto. Com o mapa completo, o grafo deve ter 120 nós e 233 arestas.

Problema 3: Caminho Mais Curto com Pesquisa em Profundidade

Um caminho válido é uma sequência de nós em que existe uma aresta entre cada par consecutivo. A distância total e o perigo total de um caminho são a soma dos pesos das suas arestas.

3a. Função objetivo

Elemento	Descrição
Função objetivo	Minimizar a distância total do caminho
Restrição principal	Perigo total do caminho ≤ limite de perigo
Restrição opcional	Distância total ≤ distância máxima

3b. get_best_path

Função auxiliar recursiva, com a mesma estrutura do enunciado original. Notas:

- O grafo tem ciclos, pelo que se deve evitar voltar a nós já visitados no caminho atual.
- Otimização obrigatória: abandonar um caminho assim que seja mais longo do que o melhor já encontrado, ou exceda o limite de perigo.
- Recomenda-se o uso de recursão.

3c. directed_dfs

`directed_dfs` devolve o caminho mais curto como lista de santuários. Se nenhum caminho cumprir as restrições, lança `ValueError`.

Testes

Os testes verificam os seguintes tipos de situação. Os resultados esperados foram obtidos com uma solução de referência; as distâncias comparam-se com uma pequena margem, por serem números decimais.

Tipo de teste	O que verifica
Caminho direto	Um só passo entre dois santuários vizinhos
Caminho longo sem limites	O caminho mais curto através do mapa
Limite de perigo apertado	A rota muda e fica mais longa, mas mais segura
Perigo zero	Só se circula por arestas seguras
Caminho impossível	Lança ValueError
Sentido das arestas	Ir de A para B pode ser diferente de ir de B para A